The Mathematics of a Dual Active-Set Quadratic Programming Solver

Thomas Schmelzer[1], Martin Stoll[2], and Enzo Montariol[3]

[1]Jebel Quant Research, Abu Dhabi
[2]Faculty of Mathematics, TU Chemnitz, Germany
[3]Affiliation to be confirmed

Abstract

This note derives the Goldfarb–Idnani dual active-set method for strictly convex quadratic programs in the form in which the `cvx-quadprog` package implements it, and proves the identities the implementation relies on. The derivation is accompanied by a numerical section that checks each identity against the running code, on the internal state the implementation actually holds, and that measures the two behaviours the derivation reasons about but cannot predict.

Three things are worth stating precisely and are not, in our experience, easy to extract from the original paper. First, the solver carries a single matrix J satisfying two invariants — $J^\top G J = I$ and $J^\top A = [R; 0]$ — and every quantity it computes is a consequence of those two lines; in particular the primal search direction is the G-orthogonal projection of $G^{-1}n_*$ onto the working set's feasible subspace, and the dual direction is the solution of a least-squares problem in the G^{-1} metric. Because both admit representation-free closed forms, any pair (J, R) meeting the invariants produces the same iterate, which is what licenses replacing the reference's chain of Givens rotations by a single Householder reflection. Second, the objective increment along a step of length t is $t\,(t/2 + u_*)\,z^\top G z$ for u_* the entering multiplier accumulated so far, an exact closed form rather than a re-evaluation, and it is strictly positive even in the reversed-step case that an equality constraint violated from above produces. Third, when the primal cannot move and no multiplier can be reduced, the vector r the solver already holds is a Farkas certificate of primal infeasibility, which we exhibit; the solver's infeasibility verdict is therefore a proof and not a failure to converge.

We then treat the two additions the package makes to the classical method: a primal–dual active-set fast path, which guesses the whole active set at once and is sound only because a strictly convex program makes the KKT conditions sufficient, so that every candidate can be certified; and warm starting across a family of programs differing only in the linear term, where J and R are reusable verbatim and a stale active set can be repaired into a dual-feasible state rather than discarded. For the general constraints treated here neither addition admits a finite-termination theorem, and we make the reason explicit: the working-set system is a principal submatrix of G^{-1} only when the constraints are bounds, and it is the loss of that P-matrix structure, not the absence of an anti-cycling rule, that removes the guarantee.

1 Introduction

The strictly convex quadratic program

$$\min_{x \in \mathbb{R}^n} \ \tfrac{1}{2} x^\top G x - a^\top x \qquad \text{subject to} \qquad C^\top x \geq b, \tag{1}$$

with $G \in \mathbb{R}^{n\times n}$ symmetric positive definite, $C \in \mathbb{R}^{n\times m}$ and the first m_{eq} of the m constraints understood as equalities, is solved a great many times a day. Mean-variance portfolio selection [8] is exactly (1); so is every step of a sequential quadratic programming method and every horizon of a linear model predictive controller. For dense instances of small to moderate size — n from a handful to a few thousand — the method of choice has for forty years been the dual active-set algorithm of Goldfarb and Idnani [2, 3], whose distinguishing property is that it needs no phase-one problem: the unconstrained minimiser $G^{-1}a$ is dual feasible for free, so the iteration can start there and drive primal infeasibility to zero.

This note is a derivation of that method in the form the `cvx-quadprog` package [11] implements it, together with the two extensions that package adds. Its aim is to state every identity the solver depends on, derive it, and name the degenerate cases — and then, in Section 10, to check each one against the code that ships, which is a different question from whether it is true. Every empirical claim made anywhere below is measured there; none is quoted from elsewhere.

1.1 Why the dual method

A primal active-set method maintains a feasible iterate and works towards stationarity. It must therefore begin at a feasible point, and finding one is a linear program in its own right — the phase-one problem. The dual method reverses the roles. It maintains stationarity and the sign conditions on the multipliers throughout, and works towards feasibility. Its starting point is free, because with no constraint active the stationarity condition reads $Gx = a$ and the multiplier vector is empty, so $x = G^{-1}a$ satisfies everything the method asks of an iterate. One Cholesky factorisation and one triangular solve replace the phase-one problem entirely.

The price is that the iterate is infeasible until the last step, so the method cannot be stopped early for an approximate answer. In exchange it terminates at an exactly feasible point of the active-set lattice rather than approaching one asymptotically as an interior-point method does, and it warm-starts almost perfectly, which is why parametric solvers are built on active-set methods [10]. Section 8 exploits that.

1.2 What is derived here

Section 2 fixes notation, records the optimality conditions, and establishes the two facts the rest of the note leans on: that (1) has at most one solution, and that its Karush–Kuhn–Tucker conditions are sufficient and not merely necessary. The second is what makes a certificate possible, and both of the extensions in Sections 7 and 8 are built on certificates.

Section 3 introduces the single matrix the solver carries and the two invariants that define it. This is the heart of the method and the part most easily mistaken for an implementation detail. The matrix is not a factor of G, nor an orthogonal factor of the constraint normals; it is the product of both, and the reason to fold them together is that it makes the search directions matrix–vector products of fixed cost rather than solves whose cost grows with the active set. We give the geometric reading — the columns are a G-orthonormal basis of $\mathbb{R}^n$ split so that the trailing block spans the working set's feasible subspace — and then show, in Proposition 3.2, that the two quantities the iteration consumes depend on the invariants alone and not on which of the many matrices satisfying them is held. That is the licence for the implementation's choice of orthogonal reduction.

Section 4 derives the iteration: the two search directions and their closed forms, the fact that the multipliers move affinely along $-r$ while the entering multiplier rises with the step, the two competing step lengths, and the exact objective increment. Section 5 collects what can be proved about stopping — strict monotonicity, termination of the inner loop in at most one pass per

active constraint, and the Farkas certificate that turns a stuck iteration into a proof of infeasibility. Section 6 derives the two factorisation updates.

Sections 7 and 8 treat the package's two departures from the classical method, and Section 9 the finite-precision questions: how the constraint-selection rule is made invariant to how each constraint happens to be scaled, and why a violation the size of the rounding in the iterate must be set aside rather than either enforced or taken as evidence of infeasibility.

1.3 Notation

Vectors are columns. For an index set $\mathcal{A} \subseteq \{1, \dots, m\}$ of size k we write $A = C_{\cdot\mathcal{A}} \in \mathbb{R}^{n\times k}$ for the matrix of the corresponding constraint normals, $b_{\mathcal{A}}$ for the corresponding right-hand sides, and c_i for the i-th column of C. Throughout, $\mathcal{A}$ is the solver's *working set*: the constraints it is currently holding as equalities. At a solution the working set is a subset of the constraints active there, and the two coincide except in the degenerate case treated in Remark 5.1. The entering constraint's normal is n_* and its right-hand side b_*. We reserve u for multipliers of working-set constraints, u_* for the entering constraint's own multiplier, and λ for a full length-m multiplier vector with zeros off the working set.

2 The problem and its optimality conditions

Write the program (1) as

$$\min_{x\in\mathbb{R}^n} \; f(x) := \tfrac{1}{2}x^\top G x - a^\top x \qquad \text{subject to} \qquad \begin{cases} c_i^\top x = b_i, & i \le m_{\text{eq}}, \\ c_i^\top x \ge b_i, & i > m_{\text{eq}}, \end{cases} \tag{2}$$

with $G \succ 0$. Two conventions are worth flagging because they differ from the textbook ones and are inherited from the reference implementation's signature. The linear term is *subtracted*, so that the unconstrained minimiser is $G^{-1}a$ rather than $-G^{-1}a$; and the constraints are held column-wise in C, so that $C^\top x \ge b$ rather than $Cx \le b$. Neither affects anything below beyond signs.

2.1 Existence, uniqueness, sufficiency

Let $\Omega = \{x : c_i^\top x = b_i \ (i \le m_{\text{eq}}),\ c_i^\top x \ge b_i \ (i > m_{\text{eq}})\}$ denote the feasible set, a closed convex polyhedron.

Proposition 2.1 (Existence and uniqueness)**.** *If $\Omega \neq \emptyset$ then (2) has exactly one solution.*

Proof. Since $G \succ 0$, write $G = R_G^\top R_G$ and complete the square: $f(x) = \frac{1}{2}\|R_G(x - G^{-1}a)\|^2 - \frac{1}{2}a^\top G^{-1}a$. So f is coercive — $f(x) \to \infty$ as $\|x\| \to \infty$ — and continuous, and Ω is closed and nonempty, so a minimiser exists. It is unique because f is strictly convex: for $x \neq y$ and $\theta \in (0,1)$, $f(\theta x + (1-\theta)y) = \theta f(x) + (1-\theta)f(y) - \frac{1}{2}\theta(1-\theta)(x-y)^\top G(x-y) < \theta f(x) + (1-\theta)f(y)$, and Ω is convex, so two distinct minimisers would produce a feasible point of strictly smaller value. □

The Lagrangian of (2) is $L(x,\lambda) = \frac{1}{2}x^\top Gx - a^\top x - \lambda^\top(C^\top x - b)$, and the Karush–Kuhn–Tucker conditions are

$$Gx - a = C\lambda, \tag{3}$$

$$c_i^\top x = b_i \quad (i \le m_{\text{eq}}), \qquad c_i^\top x \ge b_i \quad (i > m_{\text{eq}}), \tag{4}$$

$$\lambda_i \ge 0 \quad (i > m_{\text{eq}}), \tag{5}$$

$$\lambda_i\,(c_i^\top x - b_i) = 0 \quad (i > m_{\text{eq}}). \tag{6}$$

Multipliers of equality constraints are unrestricted in sign, which is the one asymmetry the algorithm has to carry through every subsequent formula.

Proposition 2.2 (Sufficiency)**.** *If (x, λ) satisfies (3)–(6) then x is the unique solution of (2).*

Proof. Let $\tilde{x} \in \Omega$. By strict convexity, $f(\tilde{x}) \geq f(x) + (Gx - a)^\top(\tilde{x} - x)$, with equality only if $\tilde{x} = x$. Substituting (3),

$$(Gx - a)^\top(\tilde{x} - x) = \lambda^\top\big(C^\top\tilde{x} - C^\top x\big) = \sum_i \lambda_i\big[(c_i^\top\tilde{x} - b_i) - (c_i^\top x - b_i)\big] = \sum_i \lambda_i(c_i^\top\tilde{x} - b_i),$$

the last step by (6) for the inequalities and by $c_i^\top x = b_i$ for the equalities. Each surviving term is non-negative: for $i \leq m_{\text{eq}}$ the bracket vanishes, and for $i > m_{\text{eq}}$ both factors are non-negative by (4) and (5). Hence $f(\tilde{x}) \geq f(x)$ with equality only at $\tilde{x} = x$. □

Proposition 2.2 is the licence for everything in Sections 7 and 8. A point equipped with multipliers passing the four conditions is not merely a plausible answer to be checked further — it is *the* answer, and a method that produces candidates may therefore be as unprincipled as one likes provided each candidate is tested. The sufficiency is what makes an unguaranteed method safe rather than merely usually right.

2.2 The working-set subproblem

Fix a working set $\mathcal{A}$ of size k with normals $A = C_{\cdot\mathcal{A}}$ and suppose A has full column rank k. The *working-set subproblem* is

$$\min_x \ \tfrac{1}{2}x^\top Gx - a^\top x \qquad \text{subject to} \qquad A^\top x = b_{\mathcal{A}}, \tag{7}$$

with every constraint in $\mathcal{A}$ held as an equality regardless of its type in the original program, and every constraint outside $\mathcal{A}$ ignored.

Proposition 2.3 (Solution of the subproblem)**.** *Problem (7) has the unique solution and multipliers*

$$u = \big(A^\top G^{-1}A\big)^{-1}\big(b_{\mathcal{A}} - A^\top x_u\big), \qquad x = x_u + G^{-1}A\,u, \qquad x_u := G^{-1}a. \tag{8}$$

Proof. Stationarity for (7) reads $Gx - a = Au$, so $x = G^{-1}a + G^{-1}Au = x_u + G^{-1}Au$. Substituting into $A^\top x = b_{\mathcal{A}}$ gives $A^\top x_u + (A^\top G^{-1}A)u = b_{\mathcal{A}}$. The matrix $A^\top G^{-1}A$ is positive definite because $G^{-1} \succ 0$ and A has full column rank, hence invertible, which yields (8); the point so constructed is the unique minimiser by the argument of Proposition 2.1 applied to the affine feasible set of (7). □

We call $A^\top G^{-1}A$ the *dual Hessian* of the working set. It appears under three different names in what follows: as the matrix whose Cholesky factor the solver carries (Section 3), as the coefficient matrix of the fast path's linear system (Section 7), and as the Gram matrix of the least-squares problem the dual direction solves (Lemma 4.1). Its invertibility is equivalent to A having full column rank, and that single fact is where every rank condition in this note comes from.

The pair (x, u) of (8), extended by zeros off the working set, satisfies stationarity (3) and complementarity (6) by construction — the working-set constraints have zero slack and the rest have zero multiplier. What it need not satisfy is the sign condition (5) on $\mathcal{A}$'s inequalities, or primal feasibility (4) off $\mathcal{A}$. The dual method maintains the first and works towards the second.

Remark 2.1 (Cold start)**.** *For $\mathcal{A} = \emptyset$ the subproblem is unconstrained and (8) degenerates to $x = x_u = G^{-1}a$ with an empty multiplier vector. The sign condition holds vacuously, so this state satisfies the method's invariant with no work beyond one Cholesky factorisation. That is the whole of the phase-one problem for a dual method. The objective there is $f(x_u) = \frac{1}{2}x_u^\top Gx_u - a^\top x_u = -\frac{1}{2}a^\top x_u$, which the implementation uses as the initial value of the running objective.*

2.3 The dual problem

The method's name is earned by the following, which also explains why the objective value can be tracked as a running total and why it increases.

Proposition 2.4 (Dual function)**.** *The Lagrangian dual of* (2) *is the concave quadratic program*

$$\max_{\lambda_i \geq 0 \ (i > m_{\mathrm{eq}})} \ \psi(\lambda) := -\tfrac{1}{2}(a + C\lambda)^{\top} G^{-1}(a + C\lambda) + b^{\top}\lambda. \tag{9}$$

If (x, λ) *satisfies stationarity* (3) *and complementarity* (6)*, then* $\psi(\lambda) = f(x)$*.*

Proof. For fixed λ, $L(\cdot, \lambda)$ is a strictly convex quadratic minimised where $Gx = a + C\lambda$, i.e. at $x(\lambda) = G^{-1}(a + C\lambda)$. Writing $s = a + C\lambda$,

$$\psi(\lambda) = L(x(\lambda), \lambda) = \tfrac{1}{2}s^{\top}G^{-1}s - a^{\top}G^{-1}s - \lambda^{\top}C^{\top}G^{-1}s + b^{\top}\lambda = \tfrac{1}{2}s^{\top}G^{-1}s - s^{\top}G^{-1}s + b^{\top}\lambda,$$

using $a + C\lambda = s$, which is (9). For the second claim, (3) says $x = x(\lambda)$, so $s = Gx$ and $\psi(\lambda) = -\tfrac{1}{2}x^{\top}Gx + b^{\top}\lambda$; and (6) together with $c_i^{\top}x = b_i$ on the equalities gives $b^{\top}\lambda = \lambda^{\top}C^{\top}x = (Gx - a)^{\top}x = x^{\top}Gx - a^{\top}x$. Adding, $\psi(\lambda) = \tfrac{1}{2}x^{\top}Gx - a^{\top}x = f(x)$. □

So along any sequence of states of the form (8) the tracked objective value is simultaneously the primal objective at the subproblem minimiser and the dual objective at the current multipliers. The method is a dual ascent: Proposition 4.3 will show the value strictly increases, and Proposition 2.4 is what makes that a statement about progress towards optimality rather than an accident of bookkeeping. It also gives the sandwich $\psi(\lambda) \leq f(x^{\star}) \leq f(\tilde{x})$ for any dual-feasible λ and primal-feasible $\tilde{x}$, so the method approaches the optimal value from below — the mirror image of a primal active-set method, which approaches it from above.

3 The matrix the solver carries

Proposition 2.3 gives the subproblem solution in closed form, and one could implement the method by evaluating (8) afresh at every iteration. That costs a $k \times k$ factorisation of the dual Hessian per step, $O(nk^2 + k^3)$, and over the $O(n)$ steps a dual method takes it comes to $O(n^4)$. The whole point of the Goldfarb–Idnani formulation is to replace it with $O(n^2)$ per step, and it does so by carrying a single matrix through the iteration and updating it.

3.1 Two invariants

Let $G = R_G^{\top}R_G$ be the Cholesky factorisation with R_G upper triangular, and set $J_0 := R_G^{-1}$, again upper triangular. Then $J_0 J_0^{\top} = R_G^{-1}R_G^{-\top} = G^{-1}$ and $J_0^{\top}GJ_0 = I$. The solver's state is a matrix $J \in \mathbb{R}^{n \times n}$ and an upper triangular $R \in \mathbb{R}^{k \times k}$ satisfying

$$J^{\top}GJ = I_n, \tag{10}$$

$$J^{\top}A = \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{11}$$

Everything the iteration computes is a consequence of these two lines, so it is worth reading them slowly. Equation (10) is equivalent to $JJ^{\top} = G^{-1}$ with J nonsingular, and says that the columns of J form a basis of $\mathbb{R}^n$ orthonormal in the inner product $\langle v, w \rangle_G = v^{\top}Gw$. Equation (11) says that in that basis the working-set normals are triangular: partitioning

$$J = \begin{bmatrix} J_1 & J_2 \end{bmatrix}, \qquad J_1 \in \mathbb{R}^{n \times k}, \quad J_2 \in \mathbb{R}^{n \times (n-k)}, \tag{12}$$

the lower block of (11) reads $J_2^\top A = 0$.

At the cold start $\mathcal{A} = \emptyset$, so (11) is vacuous and $J = J_0$ serves. The updates of Section 6 maintain both invariants by post-multiplying J with orthogonal matrices — which preserves (10) exactly because $J^\top G J = I$ is invariant under $J \mapsto JW$ for $W^\top W = I$ — while restoring the triangular shape of (11).

Proposition 3.1 (What the blocks mean)**.** *Suppose* (10)–(11) *hold with R nonsingular. Then*

(i) $R^\top R = A^\top G^{-1} A$*, so R is a Cholesky factor of the dual Hessian, unique up to the signs of its rows;*

(ii) $\operatorname{range}(J_2) = \operatorname{null}(A^\top)$*, and the columns of J_2 are a G-orthonormal basis of it;*

(iii) $J_1 = G^{-1} A R^{-1}$ *and* $\operatorname{range}(J_1) = \operatorname{range}(G^{-1}A)$*, the G-orthogonal complement of* $\operatorname{null}(A^\top)$*;*

(iv) $J_1 J_1^\top = G^{-1} A (A^\top G^{-1} A)^{-1} A^\top G^{-1}$ *and consequently*

$$J_2 J_2^\top = G^{-1} - G^{-1} A \big(A^\top G^{-1} A\big)^{-1} A^\top G^{-1}. \tag{13}$$

Proof. (i) From (11), $A^\top J = [\,R^\top \;\; 0\,]$, so $A^\top G^{-1} A = A^\top J J^\top A = [\,R^\top \;\; 0\,][\,R^\top \;\; 0\,]^\top = R^\top R$. Uniqueness of the Cholesky factor of a positive definite matrix up to the signs of the diagonal entries gives the last clause, since two upper triangular matrices with the same Gram matrix differ by a left factor $\operatorname{diag}(\pm 1)$.

(ii) $J_2^\top A = 0$ places $\operatorname{range}(J_2) \subseteq \operatorname{null}(A^\top)$; the inclusion is an equality by dimension, J being nonsingular and A of full column rank. The basis is G-orthonormal because $J_2^\top G J_2 = I_{n-k}$ is the trailing block of (10).

(iii) From (10), $J^{-1} = J^\top G$ and hence $J^{-\top} = GJ$. Inverting (11), $A = J^{-\top}[R; 0] = GJ[R; 0] = GJ_1 R$, so $J_1 = G^{-1} A R^{-1}$, whose range is that of $G^{-1}A$ since R is invertible. That range is G-orthogonal to $\operatorname{null}(A^\top)$: for $v \in \operatorname{null}(A^\top)$, $\langle G^{-1}Aw, v\rangle_G = w^\top A^\top v = 0$.

(iv) Substituting $J_1 = G^{-1} A R^{-1}$ and $R^\top R = A^\top G^{-1} A$,

$$J_1 J_1^\top = G^{-1} A R^{-1} R^{-\top} A^\top G^{-1} = G^{-1} A (R^\top R)^{-1} A^\top G^{-1} = G^{-1} A (A^\top G^{-1} A)^{-1} A^\top G^{-1},$$

and (13) follows from $J_1 J_1^\top + J_2 J_2^\top = J J^\top = G^{-1}$. □

The matrix in (13) is the *reduced inverse Hessian* of the working set: it inverts G on the feasible subspace $\operatorname{null}(A^\top)$ and annihilates the complement. Carrying J therefore means carrying that object in factored form, at no cost beyond the $n \times n$ array, and it is what turns the search directions of Section 4 into single matrix–vector products.

Remark 3.1 (Why fold the two factorisations together)**.** *An alternative state is R_G together with an explicit orthogonal factor Q of the QR factorisation of $R_G^{-\top} A$. That holds the same information — J is $J_0 Q$ — and stores no more numbers. But every use of J in Section 4 would become a triangular solve followed by an orthogonal transformation, two operations where the folded form has one, and the solve is the shape that does not reach a tuned kernel. The folded form has J lose its triangularity after the first update, which is exactly the trade: n^2 storage and $2n^2$ flops per matrix–vector product, in exchange for the products being* `gemv` *at every iteration whatever the working set does.*

3.2 The representation is not unique, and does not need to be

Invariants (10)–(11) do not determine J and R. By Proposition 3.1, R is pinned only up to a left factor $S = \mathrm{diag}(\pm 1)$, whereupon $J_1 = G^{-1}AR^{-1}$ follows; and J_2 may be any G-orthonormal basis of $\mathrm{null}(A^\top)$, so it is pinned only up to a right factor V with $V^\top V = I_{n-k}$. The freedom is therefore exactly

$$(J_1, J_2, R) \longmapsto (J_1 S,\ J_2 V,\ SR), \qquad S = \mathrm{diag}(\pm 1), \quad V^\top V = I_{n-k}. \tag{14}$$

Different orthogonal reductions realise different points of that family: a chain of Givens rotations and a single Householder reflection annihilating the same vector produce factors differing in sign, and a differently ordered sequence of insertions and deletions produces a different J_2 outright. The following says this does not matter, and it is the licence for the implementation to choose its reduction on grounds of speed alone.

Proposition 3.2 (Invariance of the iterates)**.** *Let (J, R) and $(\hat{J}, \hat{R})$ both satisfy (10)–(11) for the same G and A. Then for every $n_* \in \mathbb{R}^n$ the quantities*

$$d = J^\top n_*, \qquad z = J_2 d_2, \qquad r = R^{-1} d_1 \tag{15}$$

— with $d = (d_1, d_2)$ split at k — satisfy $\hat{z} = z$ and $\hat{r} = r$. Moreover, if the two representations are related by (14) with $V = I$, the equalities hold exactly in IEEE 754 arithmetic.

Proof. By Proposition 3.1(i) and $A^\top J = [R^\top\ 0]$ we have $A^\top G^{-1} n_* = A^\top J J^\top n_* = R^\top d_1$, so

$$r = R^{-1} d_1 = R^{-1} R^{-\top} R^\top d_1 = (R^\top R)^{-1} A^\top G^{-1} n_* = \big(A^\top G^{-1} A\big)^{-1} A^\top G^{-1} n_*, \tag{16}$$

which mentions only A, G and n_*. Likewise $z = J_2 J_2^\top n_*$, since $d_2 = J_2^\top n_*$, and by (13)

$$z = \Big(G^{-1} - G^{-1} A \big(A^\top G^{-1} A\big)^{-1} A^\top G^{-1}\Big) n_*, \tag{17}$$

again free of the representation. For the last claim, $\hat{R} = SR$ gives $\hat{d}_1 = S d_1$ and $\hat{r} = (SR)^{-1} S d_1 = R^{-1} S^{-1} S d_1$, in which the only floating-point operations beyond those forming r are multiplications by ± 1; these are exact, so the computed $\hat{r}$ and r agree bit for bit. The same argument applies to $\hat{z} = \sum_{j>k} (s_j J_{\cdot j})(s_j d_j)$. □

Remark 3.2 (What the proposition does and does not settle)**.** *It settles that the substitution is exact in the sense that matters: the iteration's decisions — which constraint is most violated, whether the step is full or partial, which constraint is dropped — are functions of z and r alone, and those are functions of (G, A, n_*) alone. It does not settle that two implementations follow the same trajectory in finite precision, because the rounding* within *the reductions differs and the decisions are discontinuous functions of the computed values. That is an empirical question, and settling it properly needs two independent implementations to compare trajectories against each other. Section 10.1 settles the weaker statement that is checkable against one: that the z and r the solver computes agree with the representation-free closed forms (17) and (16) to machine precision.*

4 One iteration

The state at the top of an outer iteration is a working set $\mathcal{A}$ of size k, the factors (J, R) satisfying (10)–(11), the subproblem solution x and multipliers u of Proposition 2.3, and the running objective $f(x)$. The invariant maintained throughout is

$$(x, u) \text{ solves the subproblem (7), and } \quad u_i \geq 0 \ \text{ for every inequality } i \in \mathcal{A}, \tag{18}$$

which by the discussion after Proposition 2.3 is stationarity, complementarity and dual feasibility together. Only primal feasibility is missing, and the iteration exists to obtain it.

4.1 Choosing the entering constraint

Compute the slacks $s_i = c_i^\top x - b_i$ for every i. If $s_i \geq 0$ for all inequalities and $s_i = 0$ for all equalities, then (4) holds, (18) supplies the rest, and x is the solution by Proposition 2.2. Otherwise a violated constraint is selected as the entering constraint. The rule is the most violated one, measured relative to the norm of its own normal:

$$v_i = \begin{cases} |s_i|, & i \leq m_{\text{eq}}, \\ -s_i, & i > m_{\text{eq}}, \end{cases} \qquad i_* \in \arg\max_i \frac{v_i}{\|c_i\|}, \qquad \text{terminating when } \max_i \frac{v_i}{\|c_i\|} \leq 0, \tag{19}$$

so that an equality counts as violated in either direction while an inequality counts only when its slack is negative. The scaling is not cosmetic.

Proposition 4.1 (Selection is scale invariant)**.** *Replacing (c_i, b_i) by $(\gamma c_i, \gamma b_i)$ for some $\gamma > 0$ leaves the feasible set, the solution, and the choice made by (19) unchanged.*

Proof. The constraint $\gamma c_i^\top x \geq \gamma b_i$ has the same solution set as $c_i^\top x \geq b_i$. Its slack scales as $s_i \mapsto \gamma s_i$ and its normal as $\|c_i\| \mapsto \gamma \|c_i\|$, so the ratio in (19) is unchanged, as are the other constraints' ratios. □

Without the division, a constraint written in basis points rather than in units would be selected first merely for having been written differently, and the solver's trajectory — though not its answer — would depend on the caller's choice of units. Note that the multiplier of the rescaled constraint scales as $\lambda_i \mapsto \lambda_i / \gamma$, so the sign conditions are unaffected too.

4.2 The two directions

Write $n_* = c_{i_*}$ and $b_* = b_{i_*}$, and form d, z and r as in (15). The following lemma is the computational core of the method; everything in the rest of this section is a corollary of it.

Lemma 4.1 (Properties of the directions)**.** *With d, z, r as in (15),*

(i) $A^\top z = 0$*: moving along z preserves the working-set equalities;*

(ii) $n_*^\top z = \|d_2\|^2 = z^\top G z \geq 0$*, with equality iff $z = 0$;*

(iii) $Gz = n_* - Ar$*;*

(iv) $z = 0$ *if and only if $n_* = Ar$, i.e. iff n_* lies in the span of the working-set normals, in which case r holds its coordinates there.*

Proof. (i) $A^\top z = A^\top J_2 d_2 = (J_2^\top A)^\top d_2 = 0$ by (11).

(ii) $n_*^\top z = n_*^\top J_2 d_2 = (J_2^\top n_*)^\top d_2 = d_2^\top d_2$. For the second equality, $z^\top G z = d_2^\top (J_2^\top G J_2) d_2 = d_2^\top d_2$ by the trailing block of (10). Since J_2 has full column rank, $z = 0$ iff $d_2 = 0$.

(iii) By (13), $Gz = G J_2 J_2^\top n_* = n_* - A(A^\top G^{-1} A)^{-1} A^\top G^{-1} n_*$, and the second term is Ar by (16).

(iv) If $d_2 = 0$ then $J^\top n_* = (d_1, 0) = [R; 0] R^{-1} d_1 = (J^\top A) r$, and J is nonsingular, so $n_* = Ar$. Conversely $n_* = Ar$ gives $Gz = n_* - Ar = 0$ by (iii), hence $z = 0$. □

Part (ii) identifies z as an ascent direction for the entering constraint's slack: the slack $n_*^\top x - b_*$ grows at rate $n_*^\top z > 0$ per unit step. Part (i) says the step costs nothing in working-set feasibility. Together with the closed forms (17) and (16) the reading is:

- z is the G-orthogonal projection of the unconstrained direction $G^{-1}n_*$ onto the working set's feasible subspace $\mathrm{null}(A^\top)$. Indeed $\Pi := J_2 J_2^\top G$ is idempotent with range $\mathrm{null}(A^\top)$ and is self-adjoint for $\langle\cdot,\cdot\rangle_G$, and $z = \Pi\, G^{-1} n_*$.
- r solves a least-squares problem in the G^{-1} metric. By (16),

$$r = \arg\min_{y\in\mathbb{R}^k} \|J^\top A y - J^\top n_*\|_2 = \arg\min_{y\in\mathbb{R}^k} (Ay - n_*)^\top G^{-1}(Ay - n_*), \tag{20}$$

  the best approximation of the entering normal by the working-set normals. Its residual, mapped through G^{-1}, is z: from Lemma 4.1(iii), $z = G^{-1}(n_* - Ar)$.

Remark 4.1 (Why not call a least-squares routine)**.** *Equation (20) is a genuine least-squares problem and it is tempting to hand it to a library. The temptation should be resisted: the factor R of its normal equations is already held, so (15) costs one triangular solve, where a library call would refactorise $J^\top A$ from scratch at $O(nk^2)$ every iteration. The same remark applies to (8): the closed form is the specification, not the implementation.*

4.3 The affine path

Fix the directions and consider moving along them. Let u_* denote the entering constraint's own multiplier, which starts at 0.

Proposition 4.2 (Dual feasibility along the path)**.** *For $t \in \mathbb{R}$ define*

$$x(t) = x + tz, \qquad u(t) = u - tr, \qquad u_*(t) = u_* + t. \tag{21}$$

Then for all t: $A^\top x(t) = b_{\mathcal{A}}$, and stationarity holds for the working set augmented by the entering constraint,

$$Gx(t) - a = A\,u(t) + u_*(t)\,n_*. \tag{22}$$

Proof. The first claim is Lemma 4.1(i). For the second, stationarity at $t = 0$ reads $Gx - a = Au + u_* n_*$, and by Lemma 4.1(iii),

$$Gx(t) - a = (Gx - a) + t\,Gz = Au + u_* n_* + t(n_* - Ar) = A(u - tr) + (u_* + t)n_*. \qquad \square$$

So the entire path is stationary, with the entering multiplier rising at unit rate and the working-set multipliers falling linearly along $-r$. Complementarity holds for every constraint except the entering one, whose slack is still nonzero while its multiplier is positive; that single violation is what the step is taken to remove. Two limits on t follow immediately.

The dual limit t_1. Dual feasibility requires $u_i(t) \geq 0$ for every inequality $i \in \mathcal{A}$. Since $u_i(t) = u_i - tr_i$ and $u_i \geq 0$, the binding indices are those with $r_i > 0$, and

$$t_1 = \min\Big\{ \frac{u_i}{r_i} \;:\; i \in \mathcal{A},\ i > m_{\mathrm{eq}},\ r_i > 0 \Big\}, \qquad i_{\mathrm{del}} = \text{an index attaining it}, \tag{23}$$

with $t_1 = +\infty$ when the set is empty. Equality constraints are excluded: their multipliers are unrestricted in sign, so no step in t can make them infeasible. This is the classical ratio test, and t_1 is the largest step that preserves (18).

The primal limit t_2. The entering constraint's slack is $n_*^\top x(t) - b_* = s_{i_*} + t\, n_*^\top z$ with $s_{i_*} < 0$, so it reaches zero at

$$t_2 = \frac{|s_{i_*}|}{n_*^\top z} = \frac{|s_{i_*}|}{\|d_2\|^2}, \tag{24}$$

finite and positive whenever $z \neq 0$, and $t_2 = +\infty$ when $z = 0$ by Lemma 4.1(ii) — in which case the primal iterate cannot move at all and only the multipliers change.

The step. Take $t = \min(t_1, t_2)$.

- If $t_2 \leq t_1$ (a *full step*), the entering constraint now holds with equality. Append it to the working set with multiplier $u_* + t_2$, update the factorisation (Section 6), and return to the top of the outer loop. The new state satisfies (18): stationarity and complementarity by Proposition 4.2 together with the slack now being zero, and the sign conditions because $t_2 \leq t_1$.
- If $t_1 < t_2$ (a *partial step*), the multiplier of i_{del} reaches zero first. Take the step, drop i_{del} from the working set, downdate the factorisation, and *recompute z and r* — both change, because J and R have — keeping the same entering constraint and its accumulated u_*. This is the inner loop.
- If both limits are infinite the problem is infeasible; see Proposition 5.3.

Remark 4.2 (Equalities violated from above)**.** *An equality constraint may be violated in either direction. When $s_{i_*} > 0$ the slack must be* decreased*, so the step is taken along $-z$: formally t ranges over the negative reals, $t_2 = -|s_{i_*}|/n_*^\top z$, the ratio test in (23) runs against $-r$ rather than r, and the entering multiplier accumulates negatively. Nothing else changes; in particular Proposition 4.2 was stated for all $t \in \mathbb{R}$ precisely so that this case needs no separate treatment, and Proposition 4.3 below covers it too.*

4.4 The objective, in closed form

The running objective is not re-evaluated from x; it is advanced by an exact increment.

Proposition 4.3 (Objective increment)**.** *Along the path (21),*

$$f(x(t)) - f(x) = t\Big(\frac{t}{2} + u_*\Big)\, n_*^\top z, \tag{25}$$

which is strictly positive whenever $z \neq 0$ and $t \neq 0$ and t, u_* have the same sign (in particular whenever $t > 0$ and $u_* \geq 0$).

Proof. Expanding the quadratic, $f(x + tz) = f(x) + t\, z^\top(Gx - a) + \frac{1}{2}t^2\, z^\top G z$. For the linear term, stationarity at $t = 0$ and Lemma 4.1(i) give $z^\top(Gx - a) = z^\top A u + u_*\, z^\top n_* = u_*\, n_*^\top z$. For the quadratic term, Lemma 4.1(ii) gives $z^\top G z = n_*^\top z$. Hence $f(x + tz) - f(x) = t u_* n_*^\top z + \frac{1}{2}t^2 n_*^\top z$, which is (25). Positivity: $n_*^\top z > 0$ by Lemma 4.1(ii), and $t(t/2 + u_*) > 0$ when t and u_* share a sign and $t \neq 0$. □

Three things are worth noting about (25). It is exact, so the objective can be carried as a running total at the cost of one multiplication per step instead of an $O(n^2)$ re-evaluation of the quadratic. It is expressed in quantities the iteration already holds — $n_*^\top z$ is needed for t_2 anyway. And it handles the reversed step of Remark 4.2 without a special case: there $t < 0$ and u_* accumulates negatively, so $t/2 + u_* < 0$ and the product with t is again positive. The value therefore increases on every nonzero step, in both directions, which is the monotonicity Section 5 needs.

Algorithm 1 The Goldfarb–Idnani dual active-set method

Require: $G \succ 0$, a, C, b, m_{eq}
Ensure: The solution of (2), or a verdict of infeasibility

$R_G \leftarrow \mathrm{chol}(G)$; $J \leftarrow R_G^{-1}$; $x \leftarrow G^{-1}a$; $f \leftarrow -\frac{1}{2}a^\top x$
$\mathcal{A} \leftarrow \emptyset$; $k \leftarrow 0$; $u \leftarrow ()$
loop ▷ outer loop
  $s \leftarrow C^\top x - b$; $s_i \leftarrow 0$ for $i \in \mathcal{A}$
  choose i_* by (19); **if** none **then return** (x, f, u) ▷ optimal, by Prop. 2.2
  $n_* \leftarrow c_{i_*}$; $u_* \leftarrow 0$
  loop ▷ inner loop
    $d \leftarrow J^\top n_*$; $z \leftarrow J_2 d_2$; $r \leftarrow R^{-1} d_1$
    $t_1, i_{\mathrm{del}} \leftarrow$ ratio test (23); $t_2 \leftarrow$ primal limit (24)
    if $t_1 = t_2 = \infty$ **then stop**: infeasible ▷ Prop. 5.3
    $t \leftarrow \min(t_1, t_2)$
    $x \leftarrow x + tz$; $f \leftarrow f + t(t/2 + u_*)\, n_*^\top z$ ▷ Prop. 4.3
    $u \leftarrow u - tr$; $u_* \leftarrow u_* + t$
    if $t_2 \le t_1$ **then** ▷ full step
      $\mathcal{A} \leftarrow \mathcal{A} \cup \{i_*\}$; $u_{k+1} \leftarrow u_*$; $k \leftarrow k + 1$
      insert (Section 6.1) and **break**
    else ▷ partial step
      $\mathcal{A} \leftarrow \mathcal{A} \setminus \{i_{\mathrm{del}}\}$; $k \leftarrow k - 1$; delete (Section 6.2)
      $s_{i_*} \leftarrow n_*^\top x - b_*$
    end if
  end loop
end loop

5 Termination

Three questions are separate and are best kept so: does the inner loop stop, does the outer loop stop, and what does it mean when the iteration gets stuck.

5.1 The inner loop

Proposition 5.1 (The inner loop terminates)**.** *For a fixed entering constraint with $n_* \neq 0$, the inner loop performs at most $k + 1$ passes, where k is the size of the working set on entry.*

Proof. Every pass that does not exit performs a partial step and removes one constraint from the working set, so after at most k of them the working set is empty. With $\mathcal{A} = \emptyset$ we have $J_2 = J$ and hence, by Lemma 4.1(ii) and (10), $n_*^\top z = z^\top G z$ with $z = JJ^\top n_* = G^{-1} n_*$, so $n_*^\top z = n_*^\top G^{-1} n_* > 0$ because $G^{-1} \succ 0$ and $n_* \neq 0$. Thus $z \neq 0$, t_2 is finite by (24), and $t_1 = +\infty$ because the ratio test (23) ranges over an empty set. The step is full and the loop exits. □

The excluded case $n_* = 0$ is a column of C that is identically zero. Such a constraint reads $0 \ge b_*$: no x influences it, so it is either vacuous ($b_* \le 0$) or renders the problem infeasible outright. The implementation scores it as infinitely violated when $b_* > 0$, which routes it to the infeasibility verdict below rather than to a division by zero in (19).

5.2 The outer loop

Proposition 5.2 (Strict ascent)**.** *Every outer iteration that ends in a full step with $t_2 \neq 0$ strictly increases the objective, and hence, by Proposition 2.4, strictly increases the dual objective ψ.*

Proof. The outer iteration's total increment is the sum of (25) over its inner passes, each term of which is positive by Proposition 4.3 — the accumulated u_* and each step t share a sign throughout, by Remark 4.2 — and the final term is nonzero because $t_2 \neq 0$. □

Corollary 5.1 (Finite termination in the nondegenerate case)**.** *If every full step has $t_2 > 0$, the method terminates after finitely many outer iterations.*

Proof. A working set determines the pair (x, u) uniquely by Proposition 2.3, and hence determines the objective value. By Proposition 5.2 that value strictly increases from one outer iteration to the next, so no working set can recur. There are finitely many subsets of $\{1, \dots, m\}$, so the iteration cannot continue indefinitely; and it can only stop by the optimality test of (19) or by the infeasibility verdict. □

Remark 5.1 (Degeneracy)**.** *The hypothesis of Corollary 5.1 fails exactly when a full step of length zero occurs, which requires $s_{i_*} = 0$: a constraint recorded as violated whose slack is already zero. In exact arithmetic that means more constraints pass through x than the working set holds — primal degeneracy — and a zero step adds one of them without changing x, u or the objective, so the ascent argument goes silent and a cycle is not excluded. Goldfarb and Idnani [3] discuss this; the classical remedies are a lexicographic or least-index rule of the kind Section 7 uses for a different purpose. In finite precision the question changes character, because a slack of exactly zero is not what one observes; Section 9 describes what is done instead.*

5.3 Infeasibility is a certificate, not a failure

The interesting case is a stuck iteration: $z = 0$, so the primal cannot move, and $t_1 = \infty$, so no multiplier limits the step. The classical reading is that the dual is unbounded — by Proposition 4.2 the whole ray is dual feasible, and by (25) with $z = 0$ the value is constant, so the argument needs care — and therefore the primal is infeasible. The cleanest statement avoids the dual entirely: the vector r that the solver already holds is a Farkas certificate.

Proposition 5.3 (Farkas certificate)**.** *Suppose the iteration reaches a state in which the entering inequality constraint i_* is violated, $s_{i_*} < 0$, and*

(a) $z = 0$, and

(b) $r_i \leq 0$ for every inequality $i \in \mathcal{A}$.

Then the feasible set Ω is empty.

Proof. By (a) and Lemma 4.1(iv), $n_* = Ar = \sum_{i \in \mathcal{A}} r_i c_i$. Let $\tilde{x} \in \Omega$ be any feasible point. Splitting the working set into its equality part $\mathcal{E}$ and inequality part $\mathcal{I}$,

$$n_*^\top \tilde{x} = \sum_{i \in \mathcal{E}} r_i \, c_i^\top \tilde{x} + \sum_{i \in \mathcal{I}} r_i \, c_i^\top \tilde{x} \;\leq\; \sum_{i \in \mathcal{E}} r_i b_i + \sum_{i \in \mathcal{I}} r_i b_i \;=\; r^\top b_{\mathcal{A}},$$

where the equality members contribute $c_i^\top \tilde{x} = b_i$ exactly, and each inequality member contributes $r_i c_i^\top \tilde{x} \leq r_i b_i$ because $c_i^\top \tilde{x} \geq b_i$ and $r_i \leq 0$ by (b). On the other hand the current iterate x satisfies $A^\top x = b_{\mathcal{A}}$, so

$$r^\top b_{\mathcal{A}} = r^\top A^\top x = n_*^\top x = s_{i_*} + b_* < b_*.$$

Combining, $n_*^\top \tilde{x} < b_*$, contradicting $\tilde{x} \in \Omega$. Hence $\Omega = \emptyset$. □

The proposition says the solver's infeasibility verdict is a proof rather than an exhausted search, and it says what the proof is: the multipliers r of the entering normal in terms of the working-set normals, which the solver computed for its own purposes, are the Farkas multipliers. The reversed case of Remark 4.2 — an equality violated from above — follows by applying the proposition to the constraint $-n_*^\top x \geq -b_*$, which the equality implies.

Remark 5.2 (The two hypotheses are exactly the two infinite limits)**.** *Hypothesis (a) is $t_2 = \infty$ by Lemma 4.1(ii) and (24); hypothesis (b) is $t_1 = \infty$ by (23). So the condition the implementation tests — neither step limit is finite — is literally the hypothesis of Proposition 5.3, with one caveat: it also requires the entering constraint to be genuinely violated. In exact arithmetic that is given, since a constraint is only selected when $v_{i_*} > 0$. In finite precision it is not, and Section 9 takes up the difference.*

6 Updating the factorisation

Each iteration changes the working set by one column, and the factors must follow. Recomputing them from scratch costs a QR factorisation of $J^\top A$, $O(nk^2)$; the updates below cost $O(n^2)$ and $O(nk)$ respectively. Over the $O(n)$ iterations of a run that is the difference between $O(n^3)$ and $O(n^4)$, and it is the whole reason the factors are carried rather than rebuilt.

Both updates work the same way. The invariant (10) is preserved by post-multiplying J with any orthogonal W, since $(JW)^\top G(JW) = W^\top W = I$; the update's task is to choose W so that (11) is restored for the new working set. Because $J' = JW$ gives ${J'}^\top A = W^\top (J^\top A)$, the same $W^\top$ acts on the rows of R.

6.1 Insertion

Let the working set of size k be extended by the entering constraint, so $A' = [\, A \;\; n_* \,]$. From (11) and $d = J^\top n_*$,

$$J^\top A' = \begin{bmatrix} R & d_1 \\ 0 & d_2 \end{bmatrix}, \tag{26}$$

which fails to be triangular only in the $n - k$ entries of d_2 below its first. Choose an orthogonal $W_2 \in \mathbb{R}^{(n-k)\times(n-k)}$ with $W_2^\top d_2 = \alpha e_1$ and set $W = \operatorname{diag}(I_k, W_2)$. Then

$$(JW)^\top A' = \begin{bmatrix} R & d_1 \\ 0 & W_2^\top d_2 \end{bmatrix} = \begin{bmatrix} R' \\ 0 \end{bmatrix}, \qquad R' = \begin{bmatrix} R & d_1 \\ 0 & \alpha \end{bmatrix}, \tag{27}$$

upper triangular of order $k+1$, and the leading k columns of J are untouched because W acts as the identity on them.

Proposition 6.1 (Full column rank is maintained)**.** *The insertion is performed only after a full step, which requires $t_2 < \infty$ and hence $z \neq 0$. Then $\alpha \neq 0$, so R' is nonsingular and A' has full column rank $k+1$.*

Proof. $|\alpha| = \|W_2^\top d_2\| = \|d_2\|$, which is nonzero iff $z \neq 0$ by Lemma 4.1(ii). R' is triangular with diagonal that of R extended by α, so it is nonsingular, and ${R'}^\top R' = {A'}^\top G^{-1} A'$ is then positive definite, which forces A' to have full column rank. □

The proposition is worth pausing on: the algorithm never has to test for linear dependence among the working-set normals. The step rules exclude it. A constraint whose normal lies in the span of those already held has $z = 0$ by Lemma 4.1(iv), so its step is limited by t_1 and it can only be added after enough partial steps have removed the dependence. This is the structural advantage that Section 7 gives up.

The choice of W_2. Any orthogonal W_2 with $W_2^\top d_2 \in \mathbb{R}e_1$ will do, and by Proposition 3.2 the choice does not affect the iterates. The reference implementation uses a chain of $n-k-1$ Givens rotations, one per trailing component. A single Householder reflection achieves the same reduction:

$$W_2 = I - \frac{2}{\beta} vv^\top, \qquad v = d_2 - \alpha e_1, \qquad \beta = v^\top v, \qquad \alpha = \pm\|d_2\|. \tag{28}$$

That $W_2^\top d_2 = W_2 d_2 = \alpha e_1$ follows from $v^\top d_2 = \|d_2\|^2 - \alpha(d_2)_1$ and $\beta = \|d_2\|^2 - 2\alpha(d_2)_1 + \alpha^2 = 2\,v^\top d_2$, whence $W_2 d_2 = d_2 - 2v(v^\top d_2)/\beta = d_2 - v = \alpha e_1$. The reflection is symmetric, so it applies to the columns of J and to d_2 as one operation, and its application to a block is a matrix–vector product followed by a rank-one update — two operations of $O(n(n-k))$ each, where the Givens chain is $n-k-1$ operations of $O(n)$ each. The arithmetic is comparable; the number of operations is not.

Remark 6.1 (The sign of α, and cancellation)**.** *Numerical analysis prescribes $\alpha = -\mathrm{sign}((d_2)_1)\|d_2\|$, which makes $v_1 = (d_2)_1 - \alpha$ a sum of like-signed terms. The implementation takes the opposite sign, $\alpha = \mathrm{sign}((d_2)_1)\|d_2\|$, so as to reproduce the sign convention of the reference's Givens chain — and thereby walks into the cancellation the prescription exists to avoid, severely when d_2 is already close to a positive multiple of e_1. The cure is algebraic rather than a change of convention: with $\tau = \|(d_2)_{2:}\|^2$ and $\nu = \|d_2\| = \sqrt{(d_2)_1^2 + \tau}$,*

$$v_1 = (d_2)_1 - \mathrm{sign}((d_2)_1)\,\nu = \frac{(d_2)_1^2 - \nu^2}{(d_2)_1 + \mathrm{sign}((d_2)_1)\nu} = \frac{-\mathrm{sign}((d_2)_1)\,\tau}{|(d_2)_1| + \nu}, \tag{29}$$

in which every operation combines like-signed quantities. The remaining entries of v are those of d_2, and $\beta = \tau + v_1^2$. By Proposition 3.2 the sign convention is anyway immaterial to the iterates; (29) is what makes it immaterial to the accuracy as well. See [4, 5] for the standard treatment.

6.2 Deletion

Let the constraint in position p of the working set be removed. Deleting column p of A deletes column p of R and shifts columns $p+1, \dots, k$ one place left, each retaining its own entries. The result is upper triangular except for one subdiagonal: writing $\tilde{R}$ for the shifted $k \times (k-1)$ matrix, $\tilde{R}_{ij} = 0$ for $i > j+1$, with the offending entries $\tilde{R}_{j+1,j}$ for $j = p, \dots, k-1$.

These entries lie in distinct columns, and no single reflection annihilates components of distinct vectors. What restores the shape is a *chase* of $k-p$ plane transformations: for $j = p, \dots, k-1$ in order, a 2×2 orthogonal acting on rows j and $j+1$ annihilates $\tilde{R}_{j+1,j}$, and the same transformation is applied to columns j and $j+1$ of J. The chase is inherently sequential — the parameters of each transformation are read from entries the previous one wrote — which is why no batched form of it exists.

The implementation takes the 2×2 block to be the symmetric reflection

$$W_j^{(2)} = \begin{bmatrix} c & s \\ s & -c \end{bmatrix}, \qquad c = \frac{x}{h}, \quad s = \frac{y}{h}, \qquad h = \mathrm{sign}(x)\sqrt{x^2 + y^2}, \tag{30}$$

where $x = \tilde{R}_{jj}$ and $y = \tilde{R}_{j+1,j}$, so that $W_j^{(2)}(x, y)^\top = (h, 0)^\top$. Symmetry is the point. The update rule of this section applies W to the columns of J and $W^\top$ to the rows of R; a symmetric block makes those the same matrix, so one 2×2 serves both sides. A Givens rotation, which is what BLAS offers, is not symmetric: it agrees with (30) on the first output and negates the second, so it requires its transpose on the R side. By Proposition 3.2 either choice gives the same iterates, the difference being a sign matrix.

Remark 6.2 (The degenerate step)**.** *When $x = 0$ the formula (30) gives $c = 0$, $s = \pm 1$, and the transformation reduces to an exchange of the two rows up to sign. The implementation performs a plain exchange, $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$, which is orthogonal and symmetric and annihilates y just as well; it differs from (30) by a sign when $y < 0$, and Proposition 3.2 says that is free.*

Remark 6.3 (Where the storage cost lands)**.** *R is held as packed columns — entry (i, j) with $i \le j$ at offset $j(j+1)/2 + i$ — so that its leading $k \times k$ block, which the triangular solve for r reads once per iteration, is a contiguous run of $k(k+1)/2$ values at every k. Insertion writes one column, which is contiguous in that layout. Deletion mixes two* rows *across a range of columns, and since column offsets grow with the column index, a row is a strided gather rather than a slice. The asymmetry is deliberate: it buys a contiguous triangular solve once per iteration at the price of a strided gather per deletion step. That trade is a property of the memory layout rather than of the derivation, and the mathematics is indifferent to it.*

7 Guessing the active set instead of walking to it

The dual method reaches the active set one constraint at a time. That is its defining feature and, at moderate n, its principal cost: a budget-plus-bounds problem in $n = 100$ variables takes some 13 outer iterations (Section 10.2), each of which is a handful of matrix–vector products. The alternative is to guess the whole active set at once, solve the working-set subproblem it induces, and repair the guess from the signs that come back. This is the semismooth Newton method of Hintermüller, Ito and Kunisch [6] read as a primal–dual active-set strategy, and block principal pivoting on the associated linear complementarity problem [7, 9, 1] read as a rule for exchanging several indices at a time.

7.1 The iteration

Given a candidate set $\mathcal{A}$, Proposition 2.3 gives the working-set solution directly:

$$\big(C_{\mathcal{A}}^\top G^{-1} C_{\mathcal{A}}\big)\lambda_{\mathcal{A}} = b_{\mathcal{A}} - C_{\mathcal{A}}^\top x_u, \qquad x = x_u + G^{-1} C_{\mathcal{A}} \lambda_{\mathcal{A}}, \qquad x_u = G^{-1} a, \tag{31}$$

with $\lambda_i = 0$ for $i \notin \mathcal{A}$. Both solves go through one Cholesky factorisation of G, computed once, and one of the $k \times k$ dual Hessian, computed per repair. The repair rule reads the two sign conditions of (5)–(4) as instructions:

$$\mathcal{A}' = \{1, \dots, m_{\text{eq}}\} \cup \big\{\, i > m_{\text{eq}} \;:\; (i \in \mathcal{A} \text{ and } \lambda_i \ge 0) \;\text{ or }\; (c_i^\top x < b_i) \,\big\}, \tag{32}$$

that is: an active constraint whose multiplier has gone negative does not belong in the set and leaves it; an inactive constraint that is violated at x does belong and joins it; equalities are always in. The starting guess is the equalities together with whatever the unconstrained minimiser violates, which is already the answer when it violates nothing. If $\mathcal{A}' = \mathcal{A}$ the conditions (4)–(6) hold to the tolerance used, and (31) supplies (3) by construction.

The whole set is exchanged at once, which is what converges in a handful of repairs almost independently of n — 1.3 to 3.5 across every family and size measured in Section 10.2. It trades arithmetic, which is abundant at these sizes, for iterations, which are not.

7.2 Why there is no finite-termination theorem here

Block exchange can cycle, and the standard remedy is a least-index rule — flip only the lowest-indexed offending constraint, in the manner of Bland's rule for the simplex method and Murty's least-index rule for linear complementarity problems [9]. In the bound-constrained case that remedy comes with a theorem, and it is worth being precise about why the theorem does not survive the generalisation, because the reason is structural rather than a gap in the argument.

Consider $\min_{x\geq 0} \frac{1}{2}x^\top Gx - a^\top x$, so that $C = I$ and $m = n$. The system solved on a candidate set $\mathcal{A}$ is then

$$\left(I_{\cdot\mathcal{A}}^\top G^{-1} I_{\cdot\mathcal{A}}\right)\lambda_\mathcal{A} = \left(G^{-1}\right)_{\mathcal{A}\mathcal{A}}\lambda_\mathcal{A} = b_\mathcal{A} - x_{u,\mathcal{A}}, \tag{33}$$

whose coefficient matrix is a *principal submatrix* of G^{-1}. Since $G^{-1} \succ 0$, every principal submatrix of it is positive definite, so G^{-1} is a P-matrix: every principal pivot is well defined, whatever the guess. That is the hypothesis under which block principal pivoting with a least-index guard terminates finitely at the unique solution [7, 9], and the guarantee survives inexact inner solves once their residual is small against the problem's decision margin [12].

None of it is available for general C. The matrix $C_\mathcal{A}^\top G^{-1} C_\mathcal{A}$ is positive definite exactly when $C_\mathcal{A}$ has full column rank, and that is a property of the guess, not of the data: a guessed set may name $k > n$ constraints, or $k \leq n$ linearly dependent ones, and then the pivot is undefined rather than merely awkward. There is no P-matrix to appeal to, and correspondingly no theorem to inherit. Contrast Proposition 6.1: the dual method's step rules make dependence unreachable, and it is exactly that protection which guessing forfeits.

The implementation's response is to treat the Cholesky factorisation of $C_\mathcal{A}^\top G^{-1} C_\mathcal{A}$ as the rank test — a dependent guess fails there rather than returning something plausible — and to treat the least-index rule as a way of making progress where block exchange oscillates rather than as a termination proof. Neither is a substitute for the guarantee. The certificate is.

7.3 The certificate

Because the method may fail, and may fail by stabilising on a set that is simply not optimal, the trade is sound only because the answer is checked. This is where Proposition 2.2 earns its place: for a strictly convex program the KKT conditions are sufficient, so a candidate (x, λ) satisfying

$$\|Gx - a - C\lambda\|_\infty \leq \varepsilon, \quad |c_i^\top x - b_i| \leq \varepsilon \ (i \leq m_{\text{eq}}), \quad c_i^\top x - b_i \geq -\varepsilon, \ \ \lambda_i \geq -\varepsilon, \ \ |\lambda_i(c_i^\top x - b_i)| \leq \varepsilon \tag{34}$$

for the inequalities, with ε scaled by the data, *is* the answer up to that tolerance — not a plausible candidate awaiting a second opinion. Stationarity is checked against G directly rather than trusted from the construction, since the construction is precisely what an ill-conditioned working set corrupts. A candidate failing (34) is discarded and the exact walk of Algorithm 1 is run instead, so the guess can never degrade the answer: it either produces the minimiser or produces nothing.

Remark 7.1 (Two tolerances with very different jobs)**.** *The tolerance in* (32) *decides which set is tried next. A bad choice there costs a repair or a fallback and can never cost correctness, so it is not delicate. The tolerance in* (34) *is the only thing standing between a non-optimal point and the caller, and is therefore the one to reason about. It is nonetheless not delicate either, because the quantity it thresholds is bimodal: a candidate constructed from a well-conditioned working set satisfies* (34) *to*

a few multiples of the machine epsilon, and one built on a set that is not has no reason to come close. Section 10.2 measures the accepted population — worst residual 1.8×10^{-13} *— and reports what it did and did not observe on the rejecting side.*

8 A family of programs differing only in the linear term

An efficient frontier, a rolling rebalance and a scenario grid all solve the same program repeatedly with a slightly different linear term: G, C, b and m_{eq} are fixed and only a varies. Solved independently, each instance rediscovers an active set it almost always already had.

What makes this exploitable is a reading of the invariants (10)–(11): *neither factor depends on* a. J depends on G alone through $J^\top G J = I$ and on the working set through the orthogonal transformations accumulated into it; R depends on G and the working set. The linear term enters only through $x_u = G^{-1}a$ and the right-hand sides. So across such a family both factors are reusable verbatim, and the entire solution can be recovered from them.

8.1 Recovery

Proposition 8.1 (Recovery from cached factors)**.** *Let* (J, R) *satisfy* (10)–(11) *for a working set* $\mathcal{A}$ *of size* k *with normals* A*. For a new linear term* a*, set*

$$x_u = J\,(J^\top a), \qquad \rho = b_{\mathcal{A}} - A^\top x_u, \qquad R^\top y = \rho, \qquad x = x_u + J_1 y, \qquad R\,u = y. \tag{35}$$

Then (x, u) *is the solution and multiplier pair of the working-set subproblem* (7)*.*

Proof. $x_u = JJ^\top a = G^{-1}a$ by (10). By Proposition 3.1(i), $u = R^{-1}y = R^{-1}R^{-\top}\rho = (R^\top R)^{-1}\rho = (A^\top G^{-1}A)^{-1}\rho$, which is the multiplier of (8). For the iterate, $Ru = y$ gives $J_1 y = J_1 R u = G^{-1}Au$ by Proposition 3.1(iii), so $x = x_u + G^{-1}Au$, again as in (8). □

The cost is worth stating exactly, since the recovery is the point. Forming $x_u = J(J^\top a)$ is two matrix–vector products with an $n \times n$ matrix, $O(n^2)$ — and no cheaper, J having lost its triangularity at the first update. Everything after it is $O(nk + k^2)$: one gather or product for $A^\top x_u$, two triangular solves of order k, and one product with J_1. Against that, re-deriving the factors for the same working set costs k insertions at $O(n^2)$ each, and a cold solve costs that plus the walk. So the saving is a factor of order k, and it is largest exactly where it matters — a long-only portfolio optimum is a vertex at which most of the bounds bind, so k is large.

Whether (x, u) from (35) is the answer to the *full* program is then one certificate away, and it is the same certificate as before: the multipliers of the working-set inequalities must be non-negative and no constraint outside the working set may be violated. By Proposition 2.2 that is a proof. Note the asymmetry in cost: the recovery is $O(n^2)$ but the check must look at every constraint, so it is $O(nm)$ for a dense C — verification, not recovery, is what dominates a successful reuse on a dense problem.

8.2 Repair rather than restart

When the check fails, the cached working set is stale, but it remains a far better starting point than the unconstrained minimum. What blocks resuming from it is (18): the iteration requires dual feasibility, and a stale set typically has one or more negative multipliers. Those are precisely the constraints that no longer belong.

Proposition 8.2 (Repair terminates in a resumable state)**.** *Iterate the following from the cached set: recover* (x, u) *by* (35)*; if every inequality multiplier is non-negative, stop; otherwise drop one constraint with a negative multiplier, downdate* (J, R) *by Section 6.2, and repeat. The loop stops after at most* k *passes, in a state satisfying* (18)*.*

Proof. Each pass that does not stop reduces the working-set size by one, so there are at most k of them; with an empty working set the multiplier vector is empty and the sign condition is vacuous, which is the cold start of Remark 2.1. On exit, (x, u) solves the subproblem for the current set by Proposition 8.1 and the sign conditions hold, which is (18). Full column rank of the remaining normals is inherited: a submatrix of a full-column-rank matrix formed by deleting columns has full column rank. □

From such a state the iteration of Section 4 cannot tell a resumed solve from a cold one — the invariant is all it reads — so the resumed walk needs only to drive the remaining primal infeasibility to zero. In the worst case everything is dropped and the resumed walk is a cold solve, so the repair is never worse than restarting up to the cost of the drops themselves.

Remark 8.1 (Why this cannot change the answer)**.** *Neither the reuse nor the repair introduces an approximation. The reuse returns a point only when it carries a certificate, and the repair merely chooses a different starting state for an iteration whose output is determined by Proposition 2.2. What does change is the reported iteration count, which counts the working-set edits actually performed and is therefore not comparable with a cold solve's.*

9 What finite precision changes

Every proposition above is a statement about exact arithmetic. Three of them are sensitive to rounding in ways worth naming, because the implementation's answer to each is a design decision rather than a tolerance to be tuned.

9.1 The scale of the arithmetic

The reference implementation fixes its working precision by a construction due to Powell: double a small positive number until it perturbs 1 when scaled by both 0.1 and 0.2. The result, call it ϵ_0, is a small multiple of the machine epsilon obtained without assuming what the arithmetic is. It plays two roles below.

9.2 Slacks that are zero, and slacks that are nearly zero

A constraint held in the working set has slack exactly zero in exact arithmetic, by construction. Computed as $c_i^\top x - b_i$ it does not, and the error is inherited from x rather than generated by the dot product: it is of order $\epsilon_0 \|c_i\| \|x\|$, which for a constraint the iterate sits on is all there is. The implementation therefore forces the slacks of working-set constraints to zero rather than computing them, and snaps to zero any slack below ϵ_0 in magnitude. Both are safeguards against selecting, and then trying to enforce, a constraint that is already satisfied.

Neither safeguard admits a principled threshold, and it is worth being clear about why. The selection rule (19) has to separate "violated by rounding" from "violated but only slightly", and no margin does that: a genuine violation can be arbitrarily small. The snap is therefore a heuristic, and the cost of getting it wrong is a wasted outer iteration, not a wrong answer — the outer loop re-measures every slack at the new iterate and will select the constraint again if it is genuinely violated.

9.3 A stuck iteration that is not a proof

Proposition 5.3 requires the entering constraint to be genuinely violated. Drop that hypothesis and the conclusion fails outright: $z = 0$ with no limiting multiplier says only that the iterate cannot move and no multiplier can be reduced, which for a constraint that is in fact satisfied is not a contradiction but a description of a point that is already fine.

This is reachable, and not merely in principle. The choice of orthogonal reduction in Section 6.1 is free by Proposition 3.2 but not free of rounding: a Householder reflection and a Givens chain leave the iterate at slightly different points, so a constraint that one implementation leaves a few multiples of ϵ_0 *inside* its boundary, the other can leave a few multiples *outside* — on either side of the fixed snap of the previous subsection. Reaching the stuck state with such a constraint selected, the solver must neither enforce it (there is nothing to enforce) nor conclude infeasibility (the conclusion does not follow).

What it does instead is set the constraint aside for the current iterate and let the outer loop take the next candidate, the set-aside list being cleared whenever the iterate moves — since a violation measured against a stale iterate says nothing. The test for "indistinguishable from rounding" is

$$|s_{i_*}| \;\le\; \kappa\,\epsilon_0 \max\big(\|c_{i_*}\|\|x\|_\infty + |b_{i_*}|,\; 1\big) \tag{36}$$

for a fixed κ. Unlike the snap of the previous subsection, this threshold *is* principled, and for an instructive reason: it does not have to separate rounding from a small violation, only rounding from *provable infeasibility*. Infeasibility is macroscopic — its size is set by the geometry of the constraints, not by the arithmetic — so the two populations are separated by a dozen orders of magnitude and any threshold between them works. The scale in (36) is $\|c\|\|x\|$ rather than the size of the terms forming the dot product, precisely because the error is inherited from x.

9.4 Structure, and why it cannot change the answer

A bound constraint $x_j \ge \ell_j$ is a column of C holding a single nonzero, $c_i = \nu e_\rho$, and a box-constrained problem consists of nothing else. Three of the per-iteration quantities then degenerate to indexing:

$$c_i^\top x = \nu\, x_\rho, \qquad d = J^\top c_i = \nu\,(J_{\rho\cdot})^\top, \qquad c_i^\top z = \nu\, z_\rho, \tag{37}$$

replacing an $O(nm)$ product, an $O(n^2)$ product and an $O(n)$ dot product respectively. The identities are exact in floating point — each is a single multiplication where the general form is a sum of n terms of which $n-1$ are zero, and adding exact zeros in IEEE 754 changes nothing — so the fast path computes the same values, not merely close ones.

Two details of the detection matter mathematically. It is performed per column, not per matrix, because the case worth optimising is mixed: a mean–variance problem carries one dense budget column $\mathbf{1}^\top x = 1$ alongside $2n$ bounds, and an all-or-nothing test would observe the dense column and send the whole problem down the general path. And an all-zero column must not be mistaken for a unit column: it expresses $0 \ge b_i$, the case excluded from Proposition 5.1, and treating it as νe_ρ with $\nu = 0$ would scale a row of J by zero and lose the infeasibility verdict that Section 5 owes it.

10 Numerical experiments

Everything above this section is a proof, and proofs leave two things open that only measurement closes.

The first is whether the code satisfies the identities. A proof establishes them in exact arithmetic; the implementation reaches them through machinery the derivation does not mention — a triangular

factor held as packed columns addressed by hand, orthogonal updates applied by library calls that write in place into a view of J's own buffer, correct only while that view stays contiguous. A sign error in the packed indexing, or a rank-one update handed a strided view and quietly given a copy to modify, would leave every proposition above true and the running solver wrong. Nothing in the derivation can detect that, and the invariants are exactly the kind of property that decays quietly: they are maintained across a run rather than recomputed, so an error does not announce itself but accumulates, and a solver that has drifted from $J^\top G J = I$ still returns a vector that is very nearly right.

The second is the set of quantities this paper reasons about but cannot predict. Section 7 argues that guessing the active set is unguaranteed but sound; how often the guess is certifiable, and how many repairs it takes, are questions about behaviour. Section 8 derives a cost; what a warm-started solve costs is not the same claim.

All measurements below are on the implementation [11] at the version recorded there, and are produced by scripts distributed with this paper's sources. Where a claim in an earlier section rests on one of them, it points here.

10.1 The identities hold in the running code

Every identity stated in this paper is evaluated on both sides against the shipped implementation, on the internal state that implementation actually holds — its J, its packed R, and the d, z and r its own routine returns — rather than against a reimplementation written for the purpose. Reaching into the private interior is the point: the claims are about objects that are internal by design, and a check written against the public surface could not see them.

Residuals are reported in the scaled sup-norm

$$\mathrm{rel}(P, Q) = \frac{\|P - Q\|_\infty}{\max(1, \|Q\|_\infty)}, \tag{38}$$

so that figures are comparable across identities of different magnitudes. The sweep runs over problems of size $n \in \{12, 30, 60, 120\}$, several seeds each, driving the factorisation into every intermediate working set reached by inserting columns one at a time: 560 distinct factorisation states over 32 problems, and 29696 individual checks.

The worst residual anywhere in Table 1 is 5.2×10^{-15}. That is the scale to expect: these are exact-arithmetic identities evaluated in double precision on matrices of condition number $O(n)$, so a few multiples of the machine epsilon is agreement, not approximation. No threshold appears anywhere in the procedure — the residuals are reported, not tested against a bound.

Two rows are exactly zero, which is a stronger statement than a small residual.

The sign conditions are never violated. The multipliers of active inequalities at the returned point are non-negative in every instance, with no negative excursion at any magnitude. That is what one wants from Proposition 2.2: dual feasibility is maintained combinatorially, by the ratio test (23) refusing to step past a zero crossing, rather than restored afterwards by clipping. A method that enforced it by clipping would show a small nonzero here.

Rescaling a constraint changes the trajectory not at all. Proposition 4.1 claims that the choice made by the selection rule (19) is invariant under $(c_i, b_i) \mapsto (\gamma c_i, \gamma b_i)$. The measured statement is stronger: over three decades of γ either side of unity, the solver performs an identical number of additions and removals and returns a minimiser agreeing to 3.3×10^{-16}. The invariance

Table 1: Worst relative residual (38) for each of the 28 identities, over 29696 checks on 560 factorisation states. Two rows are exactly zero; see the text.

Result	Identity	Worst residual	Checks
(10)	$J^\top G J = I$	2.4×10^{-15}	560
(11)	$J^\top A = [R; 0]$	7.8×10^{-16}	560
Prop. 3.1(i)	$R^\top R = A^\top G^{-1} A$	7.8×10^{-16}	560
Prop. 3.1(ii)	$J_2^\top A = 0$	3.6×10^{-16}	560
Prop. 3.1(ii)	$J_2^\top G J_2 = I$	2.4×10^{-15}	560
Prop. 3.1(iii)	$J_1 = G^{-1} A R^{-1}$	1.4×10^{-16}	560
(13)	$J_2 J_2^\top$ is the reduced inverse Hessian	5.6×10^{-17}	560
(16)	$r = (A^\top G^{-1} A)^{-1} A^\top G^{-1} n_*$	9.3×10^{-16}	560
(17)	z is the G-projection of $G^{-1} n_*$	8.3×10^{-17}	560
Lem. 4.1(i)	$A^\top z = 0$	1.1×10^{-16}	560
Lem. 4.1(ii)	$n_*^\top z = \|d_2\|^2 = z^\top G z$	3.3×10^{-16}	560
Lem. 4.1(iii)	$G z = n_* - A r$	1.7×10^{-15}	560
Prop. 4.2	stationarity along the path	2.2×10^{-15}	1120
Prop. 4.3	objective increment, both signs of t	5.0×10^{-15}	2240
§6.1	insertion preserves (10)	2.2×10^{-15}	560
§6.1	insertion preserves (11)	7.8×10^{-16}	560
Prop. 6.1	$\|\alpha\| = \|d_2\|$	1.1×10^{-16}	560
§6.2	deletion preserves (10)	2.4×10^{-15}	8728
§6.2	deletion preserves (11)	1.0×10^{-15}	8728
(3)	$Gx - a = C\lambda$ at the returned point	1.5×10^{-15}	64
(4)	$c_i^\top x = b_i$ on equalities	7.8×10^{-16}	32
(4)	$C^\top x \geq b$	5.6×10^{-16}	64
(5)	$\lambda_i \geq 0$ on inequalities	0	64
(6)	complementarity	5.2×10^{-15}	64
Prop. 4.1	rescaling a constraint: same minimiser	3.3×10^{-16}	64
Prop. 4.1	rescaling a constraint: same trajectory	0	64
Prop. 8.1	warm-start multipliers	2.1×10^{-15}	32
Prop. 8.1	warm-start iterate	7.2×10^{-16}	32

is therefore a property of the entire walk and not only of the selected index, which is what makes it safe for a caller to write constraints in whatever units are natural.

10.2 The guessed active set, and what the certificate catches

Section 7 makes two claims that only measurement settles: that the block exchange converges in a number of repairs that does not grow with n, and that the loss of the P-matrix property for general C is a real exposure rather than a technicality. Figure 1 addresses both over 5 constraint families, 30 instances each, at $n \in \{12, 25, 50, 100, 200\}$. The instrumentation wraps the shipped routines rather than reimplementing them, so what is counted is what runs.

On the four well-posed families — bounds, budget-plus-bounds, dense C, and C with equalities — every guess is certified at every size, and the repair count rises only from 1.3 to 3.5 across a sixteen-fold range of n, never exceeding 5. The comparison that matters is against the exact walk on the same instances: on budget-plus-bounds it takes 13 outer iterations at $n = 100$, and its count grows linearly where the repair count is very nearly flat. That is the trade Section 7 describes, measured.

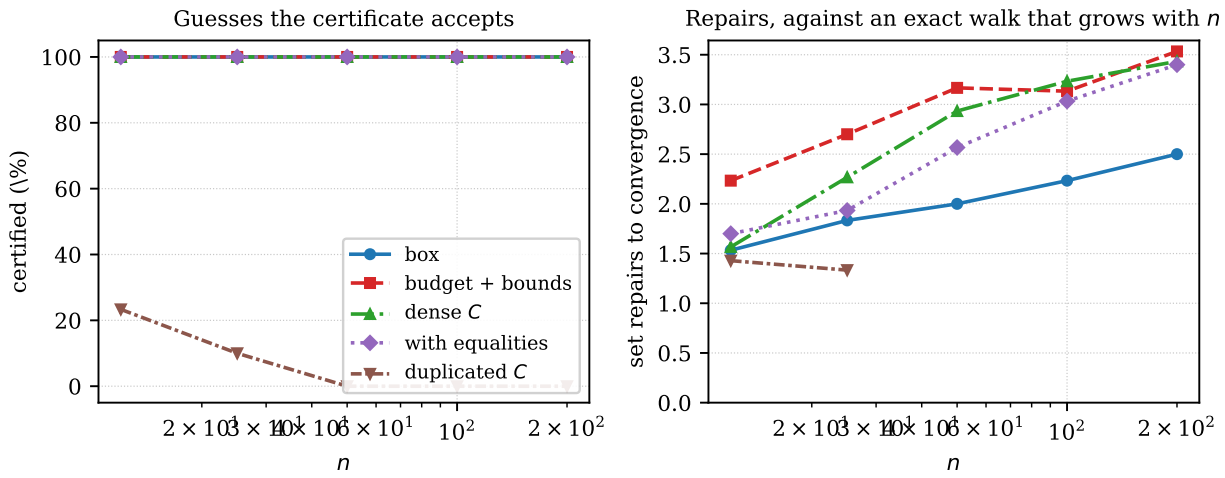

Figure 1: Left: the fraction of guesses the certificate accepts. Right: set repairs to convergence. Four families are certified without exception at every size and settle in 1.3 to 3.5 repairs while the exact walk's outer iterations grow linearly in n. The fifth is constructed so that a guessed set can be linearly dependent, and it fails — which is the point.

The rank-deficient guess is real, and it is not visible by accident. Over 30 instances at each of five sizes, not one guess on those four families produced a rank-deficient working-set system. A reader might conclude the exposure is theoretical. It is not; it is merely invisible in random data, because a random dense C almost never has a dependent subset that the guess happens to name. The fifth family makes it reachable in the most elementary way, by repeating columns of C, so that a guess naming both copies of a constraint induces a singular system. There the Cholesky rank test fires on 93.3% of attempts, and the certified fraction collapses from 23% at $n = 12$ to 0% at $n = 200$ — against 0.0%, 0.0%, 0.0% and 0.0% on the others. Duplicated constraints are not exotic; they arise whenever a model is assembled programmatically from overlapping rules. This is the concrete form of the structural point in Section 7: with bound constraints the system is a principal submatrix of G^{-1} and cannot be singular, and nothing about general C preserves that.

Where the failures are caught. Of the candidates that reached the certificate, 610 were accepted, with a worst KKT residual (34) of 1.8×10^{-13}, and 0 were rejected. We report that asymmetry rather than smoothing it: on these families the certificate never had to reject a converged candidate, because the failures were caught earlier and more cheaply, by the Cholesky factorisation of the working-set system refusing a dependent guess. The certificate remains the guarantee — it is what makes the method sound rather than merely usually right — but on this evidence it is the rank test that does the work in practice, and a reader should not infer from Section 7 that non-optimal converged candidates are common.

10.3 What a warm-started solve costs

Section 8 derives the cost of recovery from cached factors as $O(n^2)$, all of it in $x_u = JJ^\top a$, with $O(nk + k^2)$ for everything after. Measuring that requires care, because the natural experiment cannot see it: on a box-constrained family the active set grows with n, so $O(nk)$ and $O(n^2)$ predict the same curve and either reading fits. Figure 2 therefore holds the active set fixed at $k = 5$ while n grows, where the two predictions separate — doubling versus quadrupling per doubling of n.

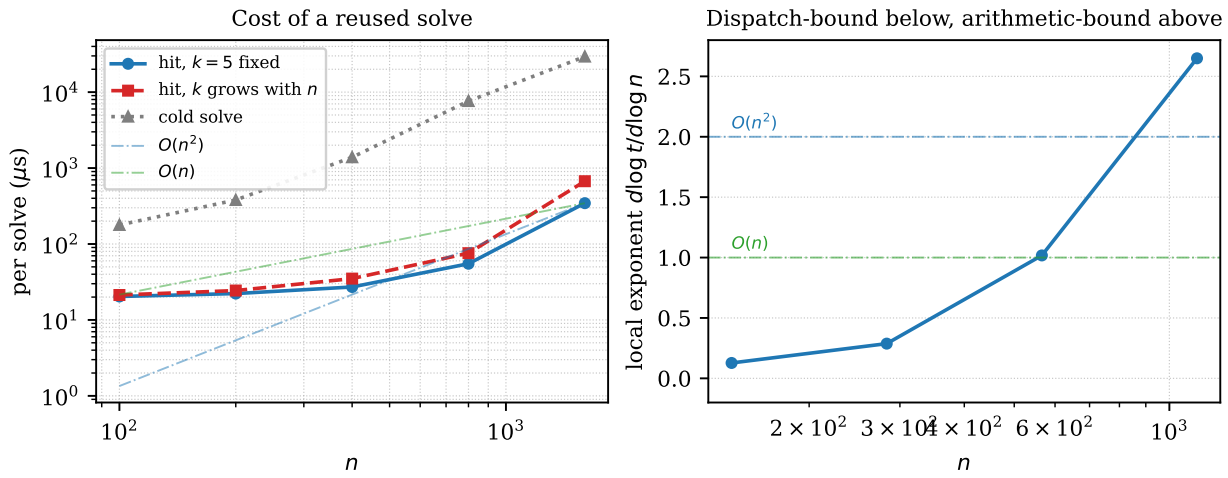

Figure 2: Left: cost per solve against n, with the active set held at $k = 5$ and with k growing, against a cold solve and two reference slopes. Right: the local exponent $d \log t / d \log n$, which is what distinguishes the two regimes.

The local exponent runs from 0.13 at the small end to 2.65 at the large end. Neither figure is 1, and the second is the one the derivation predicts: at $n = 1600$ a hit costs 345 μs against 20 μs at $n = 100$, growing faster than any linear reading allows. So the $O(n^2)$ of Section 8 is what a large warm solve costs, and an $O(nk)$ reading of the same formulas — tempting, because every term after x_u does scale with k — is wrong.

The small end is worth as much attention, because it is where the class is most used and where the flat curve invites the wrong explanation. Below roughly $n = 400$ the exponent is near zero: a hit costs what its dozen array operations cost to dispatch, not what its arithmetic costs. The flatness is an overhead floor and it ends where the n^2 overtakes dispatch. Reported as a speedup against solving each member of the family from scratch, reuse is worth 8.7× at $n = 100$ and 85.1× at $n = 1600$, and the cached set stayed optimal on 100% of solves under a relative perturbation of 0.001 to the linear term.

10.4 Against solvers that are not active-set methods

The experiments so far compare the solver with its own claims, which establishes that it is implemented correctly and says nothing about whether the method is worth using. That is a comparative question, and Table 2 and Figure 3 answer it against three alternatives: the reference C implementation of the same algorithm, an operator-splitting method (OSQP), and an interior-point method (Clarabel).

A table of times alone would be misleading here, and in a direction that flatters whichever solver was configured loosest. An active-set method terminates at an exactly feasible point of the active-set lattice and returns a KKT point to machine precision; a splitting method stops when a tolerance is met; an interior-point method approaches the boundary asymptotically and never reaches it. Every solver is therefore asked for the same accuracy, 10^{-9}, and the residual it *achieves* is reported beside its time, in the same scaled sup-norm the certificate (34) uses. Multipliers are recovered from the returned iterate by least squares rather than taken from the solver, so that no solver is charged for reporting them in another convention — the most favourable reading of each returned point.

Three things in the table are worth drawing out.

Table 2: Time per solve in milliseconds and achieved KKT residual, at $n \in \{50, 100, 200, 400\}$. The C reference row is omitted where no wheel for it is installed.

	$n = 50$		$n = 100$		$n = 200$		$n = 400$	
Solver	ms	resid.	ms	resid.	ms	resid.	ms	resid.
box								
cvx-quadprog	0.07	6×10^{-16}	0.11	1×10^{-15}	0.28	2×10^{-15}	1.14	3×10^{-15}
cvx-quadprog, `fast`	0.07	6×10^{-16}	0.08	1×10^{-15}	0.14	2×10^{-15}	0.43	3×10^{-15}
`quadprog` (C)	0.03	1×10^{-15}	0.12	8×10^{-16}	0.88	1×10^{-15}	8.27	1×10^{-15}
OSQP	0.42	1×10^{-15}	0.97	2×10^{-15}	3.47	2×10^{-15}	14.87	3×10^{-15}
Clarabel	0.56	6×10^{-16}	2.37	7×10^{-16}	8.14	1×10^{-15}	32.58	1×10^{-15}
budget + bounds								
cvx-quadprog	0.16	1×10^{-15}	0.30	2×10^{-15}	0.64	3×10^{-15}	3.47	1×10^{-14}
cvx-quadprog, `fast`	0.16	1×10^{-15}	0.24	2×10^{-15}	0.50	4×10^{-15}	1.66	5×10^{-15}
`quadprog` (C)	0.04	8×10^{-16}	0.20	1×10^{-15}	1.53	3×10^{-15}	15.35	8×10^{-15}
OSQP	0.54	1×10^{-15}	1.44	2×10^{-15}	5.58	3×10^{-15}	25.35	1×10^{-14}
Clarabel	0.64	5×10^{-8}	3.04	4×10^{-8}	9.86	2×10^{-6}	36.67	2×10^{-7}
dense C								
cvx-quadprog	0.24	5×10^{-15}	0.49	7×10^{-15}	1.52	1×10^{-14}	7.28	2×10^{-14}
cvx-quadprog, `fast`	0.13	4×10^{-15}	0.22	9×10^{-15}	0.61	1×10^{-14}	2.33	2×10^{-14}
`quadprog` (C)	0.04	5×10^{-15}	0.25	7×10^{-15}	2.41	1×10^{-14}	21.57	2×10^{-14}
OSQP	0.60	5×10^{-15}	1.85	8×10^{-15}	11.56	1×10^{-14}	67.54	2×10^{-14}
Clarabel	1.80	2×10^{-9}	5.30	7×10^{-10}	70.00	9×10^{-8}	100.80	4×10^{-5}

The same algorithm in two languages crosses over. The C reference is the fastest solver here at $n = 50$ and among the slowest at $n = 400$, where it is 7.2× slower than the implementation measured in this paper. Nothing algorithmic separates them: they walk the same active sets in the same order. What separates them is that below a few hundred variables a solve costs what its operations cost to dispatch, and above it a solve costs arithmetic, which reaches vendor-tuned kernels in one and hand-written scalar loops in the other. This reproduces, independently and as a by-product, the crossover the implementation is designed around.

Accuracy is where the interior-point method pays. Clarabel is asked for 10^{-9} and delivers between 10^{-9} and 4×10^{-5}, against 2×10^{-14} for the active-set method — several orders of magnitude, and not a defect of the solver. The optima here are vertices of the feasible polyhedron, with many constraints active, and a vertex is exactly the case an interior-point method approaches asymptotically. OSQP reaches 2×10^{-14}, comparable to the active-set method, but only with its polishing step enabled and its tolerance set to 10^{-9}; at its default tolerance it is faster and not comparable. The general point is that on combinatorially structured optima an active-set method returns an answer the others can only approach, and comparing times without residuals hides that.

Guessing the active set is worth more than the language. The `fast` path is the quickest of everything measured at $n = 400$ on every family, and it beats the exact walk by roughly a factor of two to three — more than the gap between the two implementations of the exact walk at the same size. Iterations, not the cost of an iteration, are what dominate at these sizes, which is the argument of Section 7 arriving from a different direction.

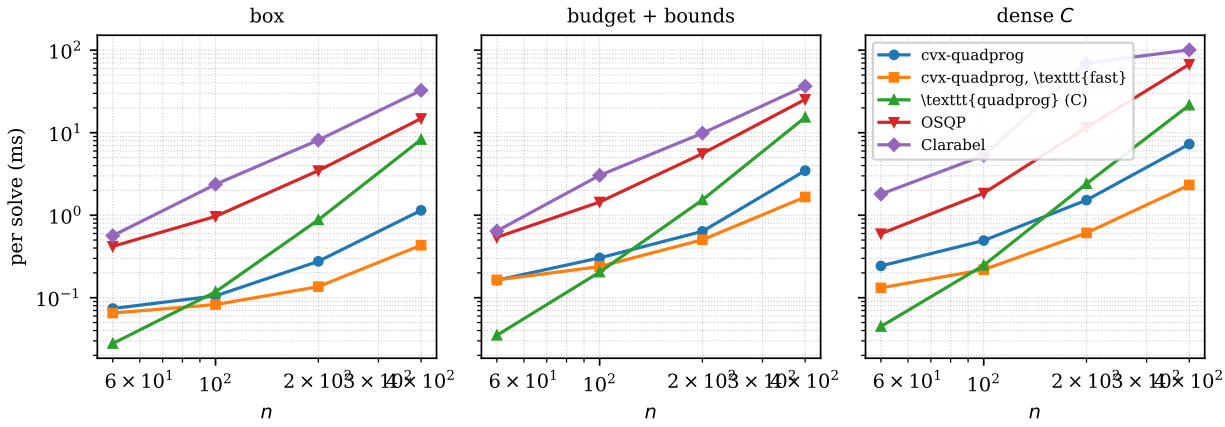

Figure 3: Time per solve against n on three constraint shapes. The reference implementation of the same algorithm wins at the small end, where cost is interpreter dispatch rather than arithmetic, and loses at the large end, where it is arithmetic.

What this comparison is not. These are dense problems of small to moderate size whose optima are vertices, which is the territory Section 1 claims for the method and precisely the territory OSQP and Clarabel are not built for. Both are designed for large sparse problems, both exploit sparsity this experiment does not give them, and both would be expected to win outright at n in the tens of thousands with a sparse G. The table is evidence that the method is the right choice on its own ground, and no evidence at all about anyone else's.

10.5 What these experiments do not establish

Three limits, stated because the section is easy to over-read.

They verify agreement between the code and the identities proved here. They do not verify the absence of defects elsewhere — in the constraint-selection scan, the infeasibility verdict, or the handling of degenerate input. A stronger instrument for that is a differential comparison of complete active-set trajectories against an independent implementation of the same method, which is not attempted here.

They say nothing about conditioning. Problems are generated with $G = BB^\top + nI$ for Gaussian B, positive definite by construction and moderately conditioned; the residuals in Table 1 would grow with $\kappa(G)$, and that table should not be read as a claim about ill-conditioned instances.

That choice also biases Section 10.2 in a direction worth naming, because it works against the conclusion drawn there. A well-conditioned G has a minimiser that spreads across coordinates, so few bounds bind and the active set is small; a realistic covariance, being close to low-rank plus diagonal, concentrates the optimum and binds many more. The exact walk's iteration count grows with the size of the set it must reach, so the counts reported here are at the low end of what an application would produce, and the advantage measured for the guessed set is correspondingly the low end of the advantage available. The direction of the bias is favourable to the method this paper does *not* advocate, which is the safe direction for it to run in, but a reader should not read 13 outer iterations at $n = 100$ as typical of a portfolio problem.

Finally, the identity checks are dense and of modest size, chosen for breadth of shape rather than scale, and the timings are single-machine. An identity that survives $n = 120$ with a working set of 60 will not fail at $n = 1200$ for mathematical reasons; if it fails there it will be for reasons of

memory layout, which the derivation is indifferent to and a timing on one host cannot settle.

11 Summary

The Goldfarb–Idnani method is often presented as a sequence of updates, which makes it look more intricate than it is. Read from the invariants it is short.

One matrix J is carried, defined by $J^\top G J = I$ and $J^\top A = [R; 0]$. The first invariant says its columns are a G-orthonormal basis of $\mathbb{R}^n$; the second says that basis is arranged so that the trailing columns span the working set's feasible subspace and the leading ones the complement. Every quantity the iteration needs is then one matrix–vector product or one triangular solve away, and every quantity it *consumes* — the primal direction z, the dual direction r — has a closed form in (G, A, n_*) alone (Proposition 3.2). That is why the representation may be chosen on grounds of speed: two implementations holding different J and R for the same working set compute the same step.

The iteration itself is one affine path (Proposition 4.2), along which stationarity holds identically while the entering multiplier rises and the working-set multipliers fall. Two limits cut it short, and which one binds decides whether a constraint is added or dropped. The objective advances by an exact closed form (Proposition 4.3) which is positive in both step directions, so the method is a dual ascent and, absent degeneracy, finite (Corollary 5.1). When the path is blocked in both directions the vector r is a Farkas certificate (Proposition 5.3): the infeasibility verdict is a proof, and the solver already holds it.

Two properties of the classical method are easy to overlook and are what the extensions of Sections 7 and 8 trade away or exploit. The first is that linear dependence among the working-set normals is unreachable: the step rules exclude it (Proposition 6.1), so no rank test is needed anywhere. Guessing the active set forfeits exactly that protection, which is the real reason a primal–dual active-set fast path admits no finite-termination theorem for general constraints — there is no P-matrix structure to appeal to once the working-set system stops being a principal submatrix. The second is that the factors depend on G and the working set but not on the linear term, which is what makes a family of programs differing only in a solvable from one factorisation, and a stale active set repairable rather than disposable.

Both extensions are unguaranteed and both are safe, for the same reason: strict convexity makes the KKT conditions sufficient (Proposition 2.2), so a candidate can be certified rather than trusted. A method that may fail, but that can recognise its own success, needs no convergence theory to be sound — it needs a fallback. That is the one idea in this note that generalises beyond quadratic programming.

References

[1] Richard W. Cottle, Jong-Shi Pang, and Richard E. Stone. *The Linear Complementarity Problem*. Academic Press, Boston, 1992. Reprinted as SIAM Classics in Applied Mathematics 60, 2009.

[2] Donald Goldfarb and Ashok Idnani. Dual and primal-dual methods for solving strictly convex quadratic programs. In *Numerical Analysis*, volume 909 of *Lecture Notes in Mathematics*, pages 226–239. Springer, 1982.

[3] Donald Goldfarb and Ashok Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming*, 27(1):1–33, 1983.

[4] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[5] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.

[6] Michael Hintermüller, Kazufumi Ito, and Karl Kunisch. The primal-dual active set strategy as a semismooth Newton method. *SIAM Journal on Optimization*, 13(3):865–888, 2002.

[7] Joaquim J. Júdice and Fernanda M. Pires. A block principal pivoting algorithm for large-scale strictly monotone linear complementarity problems. *Computers & Operations Research*, 21(5):587–596, 1994.

[8] H. Markowitz. Portfolio selection. *Journal of Finance*, 7(1):77–91, 1952.

[9] Katta G. Murty. *Linear Complementarity, Linear and Nonlinear Programming*, volume 3 of *Sigma Series in Applied Mathematics*. Heldermann Verlag, Berlin, 1988.

[10] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

[11] Thomas Schmelzer and Enzo Montariol. `cvx-quadprog`: Goldfarb/Idnani dual quadratic programming in NumPy and SciPy, 2026. Software, version 0.4.1, https://doi.org/10.5281/zenodo.22096858.

[12] Thomas Schmelzer and Martin Stoll. Non-negative conjugate gradients. Preprint, https://arxiv.org/abs/2607.22121, 2026.